Chapter 10. The Monitoring Layer

After designing and implementing the ingestion, storage, transformation, and delivery layers, the final layer to build is the *monitoring layer*. This layer is crucial for tracking and reporting on the financial data infrastructure's performance, reliability, and quality.

Data monitoring is a continuous task requiring close collaboration between financial data engineers and business teams. It enables financial institutions to operate securely, compliantly, and efficiently in a rapidly changing environment. More specifically, monitoring is needed for the following functions:

Operational continuity and efficiency

As financial data infrastructures grow in complexity, monitoring becomes vital to ensure operational continuity, system availability, efficiency, cost optimization, and optimal performance.

Compliance

Effective monitoring is crucial for financial institutions to meet regulatory requirements. It enables the detection and prevention of fraud and other suspicious activities, while also facilitating accurate and timely regulatory reporting.

Risk management

Financial institutions face a range of risks, including financial, credit, fraud, and operational risks, each with potentially significant costs. Monitoring plays a critical role in promptly and effectively detecting and mitigating these risks.

When designing this layer, the first question you should ask yourself is what components of your financial data infrastructure you need to monitor. This question is critical since monitoring can be a resource-intensive and costly commitment, so you must be clear on your monitoring plan from the start.

NOTE

Monitoring every possible issue may not be feasible, which is an important consideration when designing the monitoring layer. There is always the potential for unexpected and unforeseen problems to arise.

There is no one-size-fits-all approach to monitoring, as financial data infrastructures may differ in terms of design patterns, data management maturity, software components, level of automation, and data governance policies. However, the following five categories are applicable to (almost) all monitoring approaches: (1) metrics, events, logs, and traces, (2) data quality monitoring, (3) performance monitoring, (4) cost monitoring, and (5) business and analytical monitoring. The following sections will examine each of these in more depth.

Metrics, Events, Logs, and Traces

The main building blocks of monitoring revolve around the generation and utilization of four fundamental types of data: metrics, events, logs, and traces. These elements provide essential inputs for financial data engineers to diagnose, understand, and resolve technical and nontechnical issues within the financial data infrastructure.

Metrics

Metrics are quantitative measurements that provide information about a particular aspect of a system, process, variable, or activity. Metric values are typically observed over a specific time interval or frequency, such as daily, hourly, or in real time. In addition, metric observations are often enriched with metadata represented as a set of key-value pairs, called

tags. Tags are used to identify a given instance of a metric. A unique combination of a metric and its associated tags is called a *time series*. Table 10-1 illustrates this concept by displaying a time series view of the 30-day volatility metric across three distinct stocks traded on NASDAQ. Specifically, the table contains three separate time series: the initial two rows for AAPL-NASDAQ, the subsequent two rows for GOOGL-NASDAQ, and the final row for MSFT-NASDAQ.

Table 10-1. Metrics

Timestamp	Metric	Value	Tags
2023-07-09 09:00:00	Volatility (30-day)	0.025	Ticker: AAPL, Exchange: NASDAQ
2023-07-10 09:00:00	Volatility (30-day)	0.027	Ticker: AAPL, Exchange: NASDAQ
2023-08-11 09:00:00	Volatility (30-day)	0.020	Ticker: GOOGL, Exchange: NASDAQ
2023-08-12 09:00:00	Volatility (30-day)	0.022	Ticker: GOOGL, Exchange: NASDAQ
2023-03-11 09:00:00	Volatility (30-day)	0.014	Ticker: MSFT, Exchange: NASDAQ

Interestingly, this way of organizing metrics as time series is a key reason why many engineers prefer using time series databases for tracking and managing metrics. Common examples include Prometheus, Graphite, and InfluxDB. Advanced visualization tools like Grafana can be configured to retrieve data from the metric time series database, allowing users to explore and visualize metrics.

DATA MONITORING WITH INFLUXDB: CASE STUDIES FROM FINANCIAL MARKETS

Financial institutions worldwide employ and continually invest in data monitoring technologies. One common choice involves using time series databases such as InfluxDB.

A good example is PayPal, a global leader in the international online payments sector, serving hundreds of millions of customers in over 200 markets around the world. When PayPal decided to become container-based to modernize its infrastructure, the IT team sought a scalable monitoring solution that could unify metric collection, storage, visualization, and smart alerting within the same platform. The final solution involved using InfluxDB Enterprise (the enterprise version of InfluxDB) and InfluxData's open source plug-ins, such as Telegraf.

Another example is Capital One, a US financial institution specializing in credit cards, car loans, and other products. Capital One generates a large amount of infrastructure, application, and business process metrics. This data plays a crucial role in maintaining high-performance systems and ensuring uninterrupted service. Capital One sought an advanced monitoring solution capable of ensuring high resilience and availability across multiple regions, while also integrating seamlessly with its machine learning systems to analyze data and generate predictive metrics. To this end, Capital One created a fault-tolerant system with disaster recovery features based on InfluxDB Enterprise, AWS, and the visualization tool Grafana.

Another notable example is ProcessOut, a platform specializing in payment analytics and routing. ProcessOut provides two main products: Telescope, which analyzes transaction data to aid clients in understanding payment failures, and Smart Router, which assists clients in selecting the optimal payment system provider for specific transactions. ProcessOut chose InfluxDB to provide its consumers with the finest service while proactively monitoring and alerting them, depending on their payment actions. InfluxDB offers the functionality to collect, store, and manage critical payment information (logs, metrics, and events).

Examples like these, which you can check out online, highlight the critical role of data monitoring in the financial sector. Financial institutions prioritize monitoring not just as a necessity, but as a means to create business value and drive innovation.

Events

Events are structured data emitted by an application during runtime. They are usually triggered by a specific type of activity and tend to belong to a predefined list of possible values. Examples include client HTTP requests such as POST and GET, response status (e.g., 404 NotFound), cloud resource operation (e.g., Amazon S3 object-level API activity such as GetObject, DeleteObject, and PutObject), and user permission changes (e.g., AmazonS3ReadOnlyAccess).

The structured nature of events makes them well-suited for storage and retrieval in tabular representations. That's why event data is commonly stored in SQL databases.

Logs

Logs are semi-structured data emitted by an application during runtime, providing greater flexibility in terms of content, structure, and triggering mechanisms compared to events. Logs are typically classified into five standard levels based on the severity of the issue. These levels, arranged from lowest to highest severity, are listed here:

Debug logs

Used for diagnosing system issues in testing and development environments.

Info logs

Information on normal system operations, which can be useful in case something doesn't look normal. For example, you may want to log detailed records about transactions, including timestamps, amounts, trade submissions, account details and updates, and sta-

tus (success/failure). Such information can be critical for auditing, compliance reporting, and resolving disputes or discrepancies.

Warning logs

Information about potential issues that might lead to future problems if not addressed. These are used when something unexpected happens, but you want your application to continue running.

Error logs

Information about serious issues that affect the operation being executed but not the service or the application as a whole. They require a developer's intervention to fix. Examples include errors in processing a payment, a trade, or a loan application.

Critical logs

Information about issues affecting the entire service or application and requiring immediate intervention, for example, if the primary database of a web application becomes completely inaccessible.

The extra flexibility of logs compared to events makes them essential for uncovering the root causes of issues. Logs can be general-purpose, such as errors related to software bugs or missing resources. They can also be software-specific, such as database-related errors like deadlocks, transaction conflicts, query timeouts, and the maximum connection limits reached.

A variety of tools can be used to store and query logs. Many companies use the ELK stack: Elasticsearch, Logstash, and Kibana. Cloud-based tools are also common, including Azure Monitor Logs, Google Cloud Logging, and Amazon CloudWatch.

Traces

Traces represent a more advanced form of system behavior records, capturing comprehensive details about each step taken by a process as it progresses through one or more systems. Examples include the following:

Business operation traces

Store information describing the steps involved in completing a business operation. For instance, tracing a trade order lifecycle from submission until execution or failure; tracing a payment flow initiation, authorization, processing, validation, and settlement; and tracing a loan application from submission through credit scoring, approval, and final disbursement.

Application traces

Store information about an application's behavior as it executes multiple components. This includes function calls, API requests, condition checks, input/output, resource usage, timeout, and execution time.

Security incident traces

Store information on the events and activities associated with security incidents within a system.

UNIQUE TRANSACTION IDENTIFIERS: CORNERSTONES OF EFFECTIVE TRANSACTION TRACING

To ensure effective tracing, a trace identifier, also referred to as a transaction ID or correlation ID, is essential for uniquely identifying and monitoring the path of a particular transaction or request across a complex system. Such identifiers ensure that all related activities can be linked together, providing a complete view of the transaction's lifecycle and ensuring transparency throughout the process.

In Chapter 3, you learned about transaction identifiers such as the Unique Transaction Identifier (UTI) defined in ISO 23897. The UTI was developed to offer a consistent reference that interlinks all related messages in an end-to-end financial transaction, allowing every party involved to refer to the transaction easily. This unique identifier remains unchanged throughout the various events of a transaction lifecycle, including amendments and version changes, thereby enhancing transparency and visibility across the settlement and reconciliation value chain.

Several frameworks, technologies, and tools come into play when designing a solution for storing and managing traces. OpenLineage is an open platform designed for collecting and analyzing data lineage. It tracks events related to data pipelines, including datasets, jobs, and executions. Apache Atlas is a metadata management and data governance tool that helps with data lineage, classification, and auditing. Apache NiFi enables the automated and efficient movement of data between systems, aiding in data lineage and flow management. For distributed tracing, Jaeger and Zipkin are widely used to monitor and troubleshoot errors and identify performance and latency bottlenecks in complex, microservices-based architectures.

Data Quality Monitoring

Data generated and used by financial institutions must consistently meet high quality standards. Chapter 5 focused on financial data governance and extensively addressed the topic of data quality, discussing dimensions such as errors, duplicates, relevance, biases, completeness, timeliness, and outliers.

The traditional approach to data quality monitoring starts with the definition and development of *Data Quality Metrics* (DQMs). These metrics are quantitative measures used to assess and summarize the quality of specific aspects or dimensions of data. Defining the right DQMs depends on several factors that relate to client expectations (e.g., a data quality clause in an SLA), business requirements (e.g., complete sales and customer data), and technical standards (e.g., formatting, decimals, etc.). For this reason, there isn't a fixed list of DQMs that fit all purposes. Instead, they are defined in an iterative and continuous process that involves financial data engineers, the business team, and data analysts/machine learning experts.

While there's no one-size-fits-all approach, there are several DQM techniques that are commonly utilized. The usage of *ratios* is one such approach. For example, the *error ratio* indicates the percentage of erroneous records in a dataset, the *duplicate ratio* computes the percentage of

duplicated records compared to all records within a dataset, and a *missing values ratio* can be used to measure the rate of available data compared to a fully complete dataset. Temporal data quality metrics may also be used. For example, to ensure data timeliness, metrics can be constructed to measure the age of a data item in a dataset and compare it against a reference date (e.g., < 1 week ago). Another metric may check data arrival time against an expected arrival time (e.g., every day at 10 a.m.).

Once DQMs have been defined, a corresponding monitoring process must be implemented. A well-known approach in this regard is *data profiling*. This involves parsing and checking a given dataset to understand its content, formatting, structure, and the relationships between its records. By performing data profiling, potential data quality issues within the dataset can be identified. Profiling can be performed column-wise, where each column is examined separately, or row-wise, where all columns and their relationships are analyzed. At the end of a data profiling check, a short report can be generated to summarize the main features and issues of the input dataset and provide recommendations for addressing such issues. Figure 10-1 illustrates some of the elements a typical data profile report might contain.

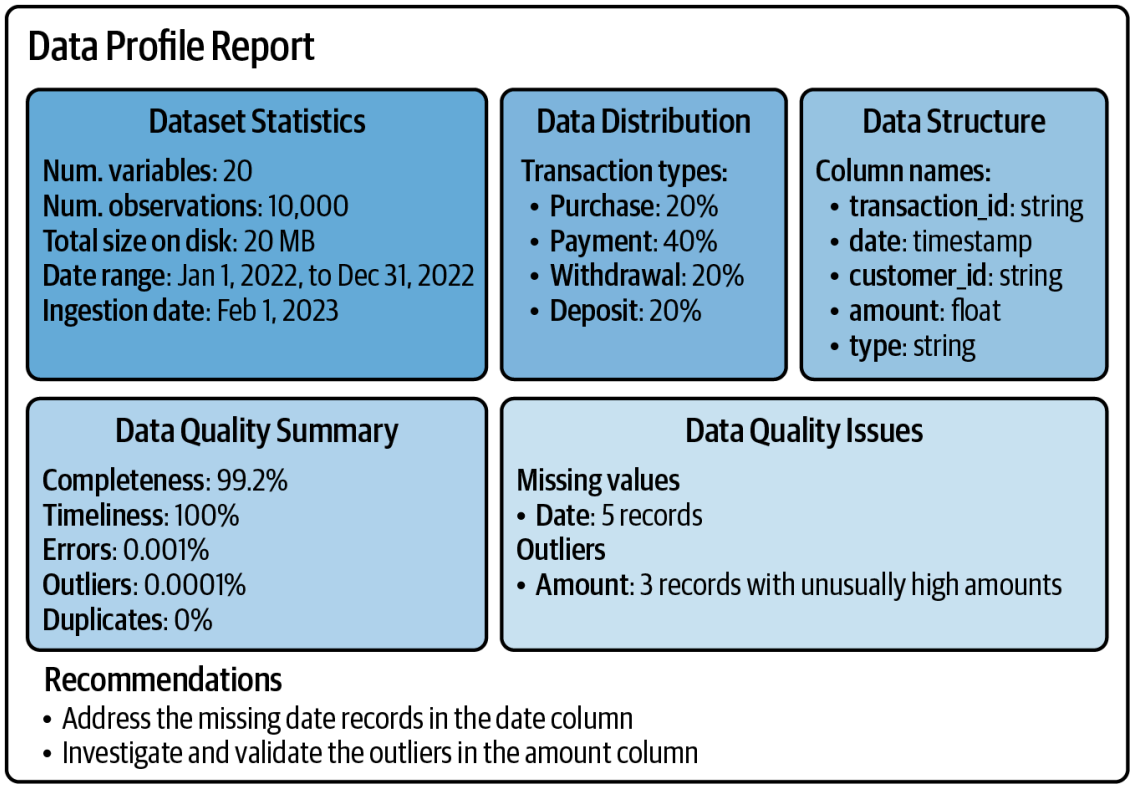

Figure 10-1. A simple example of a data profile report

Data profiling is a comprehensive procedure that scans the whole dataset to understand its data quality attributes. However, a complete data profile report may not always be required. Alternatively, a simpler approach could be to define and test data quality rules against one or more data records. A rule could state, for example, that the error ratio should not exceed 1%.

Once DQMs and the data quality monitoring process have been established, the next step is selecting a data quality tool or developing custom scripts to identify, validate, and flag any data quality issues. Commercial data quality tools include the Ataccama ONE Data Quality Suite, Informatica Data Quality, Precisely Trillium, SAS Data Quality, and Talend Data Fabric. There are also open source data quality tools such as the Python libraries Great Expectations, Soda Core, and DataCleaner.

It is essential to recognize that data quality monitoring is a continuous and evolving process. As the financial data landscape changes, with new data types, quality dimensions, and business requirements emerging, it is crucial to regularly revisit and refine your data quality monitoring process. This practice ensures the continued relevance and effectiveness of your data quality monitoring in safeguarding the quality of your data.

Performance Monitoring

Performance refers to the ability of the financial data infrastructure to meet key operational criteria such as speed, latency, throughput, availability, scalability, and resource utilization. These performance indicators directly impact business performance by influencing critical aspects such as the following:

Time to Insight (TTI)

The time it takes to obtain actionable insights from data from the point it was generated. A shorter TTI enables fast delivery of financial data to end users, ensuring efficient decision-making, customer satisfaction, timely reporting, and revenue growth.

Time to Market (TTM)

: The time it takes for a product to progress from a conceptualized idea to its introduction into the market.

Innovation velocity

: The pace at which data-driven innovations are generated and implemented

Data cycle time

: The time required to complete an entire cycle of data analysis, starting from problem formalization to obtaining actionable insights.

Various metrics are frequently used to monitor data infrastructure performance. Some are software-agnostic and apply to any type of application:

RAM usage

: The amount of memory being used by the application

CPU usage

: The percentage of CPU being used by the application

Storage usage

: The amount of disk storage being used by the application

Execution time

: The time needed to execute a task or process a request

Requests per second (RPS)

: The number of requests made to an application every second

Ingress/egress bytes (bytes/sec)

: The amount of network traffic (in bytes) entering and leaving an application

Uptime/downtime

: The ratio of time a system is operational to the total time observed

API response time

: The time it takes for an API endpoint to process and respond to a request

Concurrent users

: The number of users accessing an application simultaneously

Additionally, performance metrics can be tailored to specific software systems. For example, database management systems may have metrics such as the following:

Read/write operations

: Count of read and write operations performed on the database within a specific temporal window

Active connections

: Number of currently open database connections

Connection pool usage

: Utilization of connections within a connection pool

Query count

: Total number of queries processed by the database during a specified interval

Transaction runtime

: Duration taken to execute a database transaction

Replication lag

: The delay between the time data is written to the primary database instance and the time it is synched with read replicas

You can even have more granular metrics defined for a specific software component. For example, database queries may be monitored via metrics such as the following:

Records read

The number of data records read by the database.

Blocks read

The number of data blocks read by the database.

Bytes scanned

The total number of bytes fetched by a query.

Number of output rows

The final number of rows returned by a query.

Scan time

The time spent by the database scanning the data.

Sort time

The time spent by the database sorting the data.

Aggregation time

The time spent by the database aggregating the data.

Peak memory

The maximum amount of memory used during query execution.

Query plans

The execution steps that a data storage system follows to fetch data in response to a query. Monitoring query plans can help detect costly queries and unused DB objects such as indexes and provide insights into query performance.

Additionally, you can implement performance metrics specific to your business's technical needs. Examples from financial markets include the following:

Trade execution latency

The time taken from when a trade order is placed until it is executed, which is critical for optimizing trading strategies and minimizing execution risks.

Trade execution error rate

The frequency of failed transactions or trade orders, indicating operational inefficiencies.

Trade settlement time

The duration from when a trade is executed to when it is settled, reflecting the efficiency of the settlement process.

Market data latency

The time it takes for financial market data to be received from the exchange or data provider to the trading system, which impacts the speed of decision-making.

Algorithmic trading performance

Includes metrics such as algorithm execution time and success rate, which can highlight the effectiveness and reliability of algorithmic trading strategies.

Risk exposure calculation time

The time taken to compute and update risk exposure metrics, which is critical for managing and mitigating financial risks in real time.

Customer transaction processing time

The duration from when a customer initiates a transaction (e.g., deposit, withdrawal) to its completion, which can impact customer

satisfaction and operational efficiency.

CASE STUDY: MONITORING REAL-TIME FINANCIAL DATA FEEDS

In today's fast-paced financial markets, real-time data feeds have become increasingly popular for providing the critical information necessary for making timely decisions. These feeds deliver up-to-the-second market data, including prices, volumes, and order book information, which are indispensable for traders, financial institutions, and automated trading systems. The reliance on accurate and timely data to capture market opportunities and manage risks effectively makes financial market feeds integral to the modern financial ecosystem. To ensure efficient and timely data delivery, providers of data feeds and related infrastructure need to thoroughly monitor the performance of their systems. Among the most important data feed performance metrics to monitor are the following.

Network Metrics

Packets per second (PPS)

The number of data packets transmitted and received per second.

Packet size distribution

The size of data packets being transmitted and received.

Round-trip time (RTT)

The time for a signal to travel from sender to receiver and back.

One-way latency

Tracks the time for a signal to travel from sender to receiver.

Bandwidth utilization

Measures how much of the available network bandwidth is being used. Network bandwidth refers to the maximum rate at which data can be transmitted over a network connection in a given amount of time. Bandwidth utilization is about efficiency and capacity usage.

Data transfer rate (throughput)

Measures the actual amount of data transmitted per second. It's about the speed of data movement.

System Metrics

CPU usage

Percentage of CPU capacity used.

CPU time in I/O wait (iowait)

Time CPU spends waiting for I/O operations to complete.

CPU interrupt time (irq)

Time CPU spends handling software interrupts.

Memory utilization

Percentage of memory being used.

Processing Metrics

Backpressure

Tracks the system's ability to handle incoming data rates. Backpressure occurs if the system receives more message requests than it can handle.

Buffer/queue sizes

Monitor data waiting to be processed.

A particular type of performance metrics are incident response and management metrics, which are used to evaluate the effectiveness of an organization at tackling and preventing system downtime/outage issues. Examples include the following:

Mean Time to Detect (MTTD)

The expected time it takes to detect an issue or incident after it has occurred.

Mean Time Before Failure (MTBF)

The expected time between two failures/downtime in a repairable system. The higher the MTBF, the more reliable and available the system is, and the more effective the data engineering team is at preventing future issues.

Mean Time to Recovery (MTTR)

The expected time before a system recovers from a failure and becomes fully operational. A low MTTR indicates that the engineering team is quite effective at resolving the problem and that issue-fixing is efficient (e.g., through DevOps, DataOps, automated testing, etc.).

Mean Time to Failure (MTTF)

The expected time until a nonrepairable failure occurs. A nonrepairable failure requires replacing the failed system with a new one.

Mean Time to Acknowledge (MTTA)

The expected time from when an incident is reported to when someone starts working on the issue. A low MTTA indicates high team responsiveness and an effective alerting system.

Incident metrics are of primary importance in today's always-on financial market infrastructures. The costs associated with outages and downtime in financial markets can be substantial, potentially leading to significant repercussions, especially given the interconnected and high-frequency/high-volume nature of financial activities.

Turning to technological implementations, there are numerous tools and frameworks designed for performance monitoring. Some are integrated into data engineering tools; for example, Apache Spark includes a UI interface for monitoring cluster status and resource usage. Open source tools such as Prometheus, designed primarily for metrics data, provide a rich set of monitoring and alerting features, including a multidimensional data model, flexible query language (PromQL), and multiple modes of

dashboarding and visualizations. Another common approach involves using time series databases like InfluxDB, alongside advanced visualization tools such as Grafana.

Another popular and user-friendly option is cloud-based monitoring solutions. Using the cloud, it is possible to establish an alerting policy to get notified when performance-related issues occur. For example, Google Cloud provides a managed alerting service where you can create an *alerting policy* that defines the circumstances under which an incident is created and the notification channel through which you want to be notified.

Cost Monitoring

Cost monitoring is the process of keeping an eye on a financial data infrastructure's expenditures to ensure that they are within acceptable bounds and to spot patterns of excessive usage. The significance of cost monitoring has grown alongside the widespread adoption and migration to cloud services. While the cloud provides a plethora of accessible services with flexible on-demand pricing, managed scalability, and simplified configuration, there exists a potential risk of unforeseen, excessive, or misinterpreted cost structures.

To illustrate the issue, let's consider one of the most attractive cloud computing strategies: *serverless computing*. This term refers to a cloud-based application development model where the cloud provider takes full care of infrastructure configuration and management. This includes all features such as load balancing, auto-scaling, availability, security, operating system management, logging, monitoring, and storage management. Services such as AWS Lambda and Google BigQuery are among the most popular services that people often associate with serverless computing.

Crucially, the economics of serverless computing can be misunderstood, potentially resulting in unforeseen expenses. This is due to the variability in cost-effectiveness of the serverless model, which is dependent on factors like execution patterns and workload volume. Pennies can add up to thousands of dollars when running a massive number of jobs.

Furthermore, in services like AWS Lambda, pricing isn't solely determined by the frequency and duration of function invocations, as many might assume with *pay-as-you-go* models. You also indirectly pay based on the memory allocated to your function, which directly influences the cost of your function executions. Moreover, additional charges may be added if AWS Lambda reads data from AWS storage or transfers data between regions.

As of the time of writing this book, invoking an on-demand AWS Lambda function costs $0.0000000021 per millisecond for 128 MB of RAM and $0.0000000167 per millisecond for 1,024 MB of RAM. This looks relatively cheap. However, costs can escalate significantly based on the nature of your workload. When assessing application scalability, a commonly used metric is transactions per second (TPS), also referred to as hits per second (HPS) or requests per second (RPS). Suppose our application processes a specific number of RPS over a total of 100,000 seconds in a month. Using this information, let's explore a few scenarios:

Scenario 1: 100 RPS with a duration of 10 seconds each

- *Cost per invocation*: $0.0000000021 × 10,000 milliseconds (10 seconds) = $0.000021
- *Monthly cost*: $0.000021 × 100 RPS × 100,000 seconds = $210

Scenario 2: Increasing execution time to 1 minute (60 seconds)

- *Cost per invocation*: $0.0000000021 × 60,000 milliseconds (60 seconds) = $0.000126
- *Monthly cost*: $0.000126 × 100 RPS × 100,000 seconds = $1,260

Scenario 3: Increasing allocated memory to 1,024 MB (1 GB)

- *Cost per invocation*: $0.0000000167 × 60,000 = $0.001002
- *Monthly cost*: $0.001002 × 100 RPS × 100,000 seconds = $10,020

These calculations show how increasing execution time and memory allocation can significantly impact the monthly cost of running AWS Lambda

functions. Adjustments in these parameters should be carefully considered based on performance requirements and budget constraints. Additionally, when assessing solutions like Lambda and other alternatives, it is important to consider your future scaling needs rather than just your current usage.[1]

Various practices and tools are available to manage cloud costs. For instance, cloud providers offer budgeting options that let users set target budgets and receive notifications if costs exceed predefined thresholds. Another promising approach is FinOps, a recent trend in cloud cost monitoring and management, which has been defined by the FinOps Foundation as:

> *An operational framework and cultural practice which maximizes the business value of cloud, enables timely data-driven decision making, and creates financial accountability through collaboration between engineering, finance, and business teams.*

A FinOps framework requires five pillars:

- Cross-functional collaboration among engineering, finance, and business teams to enable fast product delivery while ensuring financial and cost control.
- Each team takes ownership of its cloud usage and costs.
- A central team establishes and promotes FinOps best practices.
- Teams take advantage of the cloud's on-demand and variable cost model while optimizing the tradeoffs among speed, quality, and cost in their cloud-based applications.
- Decisions are driven by the cloud's business value, such as increased revenue, innovative product development, reduced fixed costs, and faster feature releases.

FinOps is an iterative and learning-driven process. Within organizations, the maturity of FinOps is commonly assessed using the FinOps Maturity Model (FMM), which consists of three stages: Crawl, Walk, and Run. As an organization progresses from Crawl to Run level, it moves from being reactive, where issues are fixed as they occur, to being proactive, where

teams are able to anticipate cloud usage patterns and expenses and incorporate such information into their cloud architecture design decisions.

CASE STUDY: FINANCIAL TRANSACTION COST MONITORING WITH ABEL NOSER

Transaction cost monitoring and analysis is a common practice in financial markets. It entails monitoring and analyzing the expenses related to executing financial transactions, such as trades and investments. This process is vital for investment firms, asset managers, brokers, and other financial institutions to ensure that they are achieving optimal execution while minimizing costs.

Several companies specialize in transaction monitoring services for the financial sector. An industry leader in this market is Abel Noser, now part of Trading Technologies. Financial institutions submit their transaction data to Abel Noser, which conducts comprehensive cost analysis and returns insights to the clients. Over 350 global clients from the US, as well as other parts of the world, report their data to Abel Noser.

Abel Noser is also a data vendor, providing institutional investor datasets that contain transaction-related information such as the instrument traded, price, quantity, date, trade direction (buy or sell), commissions paid, and other market fields. Abel Noser's data is an essential source for scientific research on institutional trading.

Business and Analytical Monitoring

A more advanced form of data monitoring involves observing statistical, analytical, and business-related dimensions. This monitoring approach ensures that not only is the raw data monitored, but also the contextual and analytical insights that drive strategic decision-making.

For example, commercial banks must diligently and continuously monitor the status of their lending activities to ensure clients make timely payments and detect/predict defaulting clients. Moreover, to ensure resiliency and regulatory compliance, banks need to monitor their finan-

cial, credit, and operational risks as well as their exposures, risk concentration, capital adequacy, and leverage situation. Similarly, institutions such as investment firms need to continuously monitor their portfolio performance, asset allocation, diversification, and investment strategies.

RISK MONITORING IN FINANCIAL INSTITUTIONS: DATA ENGINEERING CHALLENGES

Risk is an inherent aspect of financial institution operations, particularly in the banking sector. Consequently, most banking regulations emphasize principles and frameworks to ensure banks' resilience against the classes of risks they are exposed to.

The primary framework for international banking regulation consists of the three Basel Accords—Basel I, Basel II, and the more recent Basel III. The Basel framework groups bank risks into three broad categories:

Market risk

> The risk of financial loss deriving from movements in market prices.

Credit risk

> The risk of financial loss deriving from a borrower or counterparty not being able to meet the agreed-upon obligations.

Operational risk

> The risk of financial loss resulting from poorly designed or failed internal processes, people, and systems or from external events.

To be managed, these risks must be quantitatively measured. To do this, risk data is needed. Each risk category requires different sets of data fields. Market risk measurement requires data such as portfolio composition, historical prices, volatility, interest rates, and correlations. Credit risk measurement requires data such as credit ratings, credit scores, exposure data, collateral data, default data, probability of default data (likelihood of creditors defaulting on their obligations), and loss given default data (losses incurred in case of a default event). Finally, operational risk measurement requires detailed operational loss and risk event data as well as internal control data.

Importantly, effectively managing the diverse types of risk data poses significant data engineering challenges related to data collection, aggregation, integration, normalization, quality assurance, and timely delivery. In

Chapter 5, we discussed the problem of data aggregation in financial institutions and illustrated how it mainly applies to risk data. As a reminder, Basel III introduced the *Principles for Effective Risk Data Aggregation and Risk Reporting* to ensure the adequacy of financial institutions' data infrastructure for managing and aggregating risk data efficiently.

To address operational risk, a common strategy involves establishing and maintaining an *operational event risk database*, which stores historical records stemming from operational incidents. This database may contain elements such as the event date, description (e.g., what happened), primary causes (e.g., human error), business line (e.g., risk management), and control point (e.g., trading desk). To learn more about this, consult the paper by Niclas Hageback, "The Operational Risk Framework Under a Retail Perspective".

Financial institutions relying on data analytics and machine learning must actively monitor the statistical, quality, and performance aspects of both their data and models. Within the machine learning community, concepts such as *concept drift* and *data drift* are frequently used in the monitoring and detection of changes in the underlying data distribution or model relationships.

Concept drift occurs when a machine learning model no longer accurately reflects the original problem it was trained to solve. For example, a well-performing fraud detection model trained on a specific financial dataset may become less effective over time if fraudsters adapt and modify their fraudulent tactics. As a result, the model's predictions may become less accurate because the underlying problem it was designed to address has evolved.

Data drift happens when the input data distribution to a machine learning model changes over time. Consider an ML model trained to forecast a company's stock price based on a given data sample. Suppose that throughout this sample, the stock price was relatively stable (low volatility). If the market becomes more volatile over time, the model might

struggle to predict accurately because the statistical characteristics of the data have changed.

Interestingly, the ideas of concept and data drift are relatively familiar to finance. In financial markets, particularly among risk managers, the term *model risk* is used to describe the risk that financial models used in asset pricing, trading, forecasting, and hedging will generate inconsistent or misleading results.[2]

One prevalent area of business monitoring among financial institutions involves the monitoring of various fraudulent and illegal activities when conducting transactions. Examples include the following:

Money laundering

The process of concealing the origins of illegally obtained money, typically by means of transfers involving foreign banks or offshore companies.

Terrorism financing

Providing financial support to terrorist organizations or individuals to facilitate acts of terrorism.

Fraud

Deceptive practices intended to secure unfair or unlawful financial gain, often involving misrepresentation or omission of information.

Corruption

Dishonest or fraudulent conduct by those in power, typically involving bribery, embezzlement, or abuse of authority for personal gain.

Another good example is market manipulation and securities fraud practices, which involve illegally influencing the supply or demand of financial instruments to gain advantage from market reactions. Examples include the following:

Pump and dump

This involves artificially inflating the price of a stock or other asset through false or misleading statements (pump). Once the price is high enough, the manipulator sells their holdings at a profit (dump), causing the price to collapse, and leaving other investors with losses.

Spoofing

This involves placing orders without the intention of executing them in order to create a false impression of demand or supply in the market. Once other traders react to these orders, the spoofer cancels or modifies their original orders.

Insider trading

Trading securities based on material nonpublic information, which can generate unfair advantages and distortions in market prices.

Front running

When a broker or trader executes orders on a security for their own account while taking advantage of inside knowledge of a future transaction that is expected to impact the security's price substantially.

Churning

Excessive trading of a client's brokerage account by a broker to generate commissions without regard for the client's investment goals.

Wash trading

Simultaneously buying and selling the same financial instruments to create artificial trading volume or a false impression of market activity without having exposure to market risk.

Lastly, an integral aspect of financial institution monitoring revolves around evaluating investment portfolio performance. This entails selecting, calculating, and monitoring key performance indicators like the

Sharpe ratio, Sortino ratio, Treynor ratio, and information ratio. To illustrate one example, the Sharpe ratio is a measure of risk-adjusted return, indicating how well an investment performs relative to its risk level. These metrics allow institutions to assess the effectiveness of their investment strategies, enabling informed and timely decisions for optimizing portfolio composition and performance.

Data Observability

As the technological stacks supporting data infrastructures have grown in complexity and scale, the requirements for monitoring have evolved accordingly. This has led to the emergence of a more advanced and comprehensive monitoring approach known as *observability*.

Observability elevates monitoring to a new level, allowing software and data engineering teams to handle issues proactively and gain deep insights into the internal state and behavior of a given software application or infrastructure. Authors Charity Majors, Liz Fong-Jones, and George Miranda (*Observability Engineering*, O'Reilly, 2022) describe a software system as observable if you can do the following (the following list is a direct quote):

- Understand the inner workings of your application
- Understand any system state your application may have gotten itself into, even new ones you have never seen before and couldn't have predicted
- Understand the inner workings and system state solely by observing and interrogating with external tools
- Understand the internal state without shipping any new custom code to handle it (because that implies you needed prior knowledge to explain it)

Observability can be applied across various dimensions of IT systems, encompassing data, applications, infrastructure, networks, security, and business. *Data observability* is a vital area in this regard. Author Andy

Petrella of *Fundamentals of Data Observability* (O'Reilly, 2023) defines data observability as:

> *The capability of a system to generate information on how the data influences its behavior and, conversely, how the system affects the data.*

With data observability, financial data engineers and other team members should easily be able to ask and answer questions such as: "Why is workflow A running slowly?", "Why is data not available yet for client X?", "What caused the data quality issue in dataset Y?", "Why is the data pipeline for report Z delayed?", or "Where is the bottleneck in a transformation or ingestion process?"

The main building blocks of data observability are metrics, events, logs, and traces.

In addition, data observability systems leverage concepts such as automated monitoring and logging, root cause analysis, data lineage, contextual observability, Service-Level Agreements (SLAs), telemetry/OpenTelemetry, instrumentation, tracking, tracing, and alert analysis and triaging.

OPENTELEMETRY: THE OPEN SOURCE STANDARD FOR OBSERVABILITY

OpenTelemetry is an open source, vendor-neutral framework comprising a set of APIs, SDKs, libraries, agents, and tools to enable observability in software applications. It allows engineers to collect telemetry data, such as traces, metrics, and logs, from their applications and services to gain insights into their behavior, performance, and reliability. OpenTelemetry provides standardized instrumentation and integration capabilities for various programming languages, frameworks, and platforms, making it easier to instrument applications consistently across different environments. It allows exporter applications to send telemetry data to various backends and observability platforms for storage, analysis, visualization, and alerting.

A properly implemented data observability system can bring a lot of benefits for financial institutions, such as the following:

- Higher data quality (e.g., by observing data quality metrics)
- Operational efficiency (e.g., reduced TTD and TTR)
- Facilitated communication and trust between different team members (e.g., risk management and trading desk)
- Gains in client trust by being able to detect and understand the issues they might face
- Enabling complex data ingestion and transformation while maintaining reliability and visibility into the system
- Regulatory compliance (e.g., by keeping data privacy and security under control)

Financial data engineers play a vital role in embedding data observability capabilities within the financial data infrastructure. Observability is fundamentally a data engineering challenge. It entails instrumenting systems to generate vast amounts of heterogeneous data points, which must be efficiently indexed, stored, and queried in near-real time. This ensures comprehensive visibility into the behavior of various components of the financial data infrastructure, including data ingestion, storage, processing systems, workflows, quality, compliance, and governance.[3]

It's important to remember that implementing a data observability system requires a significant investment in both technology and people. For this reason, I highly recommend you evaluate your business needs, technical constraints, and growth perspectives before investing in a large-scale data observability system. In many cases, a simple monitoring strategy is all you need. As your organization grows and its technological stack becomes more complex, investing in data observability becomes increasingly beneficial and, eventually, essential.

Summary

This chapter discussed the final layer in the financial data engineering lifecycle: monitoring. Monitoring is critical for guaranteeing the reliabil-

ity, performance, security, and compliance of financial data infrastructures.

This chapter explained and illustrated the significance of metric, event, log, and trace monitoring in tracking application activities and diagnosing and solving potential issues. Furthermore, it explored the key components of data quality, performance, and cost monitoring, outlining their techniques and giving practical advice and financial use cases.

In addition, it emphasized the importance of business and analytical monitoring in providing actionable insights and supporting informed decision-making within financial institutions. Finally, the chapter included a brief review of data observability, an emerging topic, highlighting its crucial role in providing deep insights into the internals and behavior of various data infrastructure components.

By this point of the book, you should have a solid grasp of the various layers comprising the financial data engineering lifecycle. To further simplify and enhance the modularity of financial data infrastructures, smaller data processing components, known as data workflows, are commonly developed. The next chapter will cover these data workflows in detail.

1 To learn more about the economics of serverless computing, I highly recommend Adam Eivy and Joe Weinman's "Be Wary of the Economics of 'Serverless' Cloud Computing", *IEEE Cloud Computing* 4, no. 2 (March–April 2017): 6–12.

2 For more on model risk, see Jon Danielsson, Kevin R. James, Marcela Valenzuela, and Ilknur Zer's "Model Risk of Risk Models", *Journal of Financial Stability* 23 (April 2016): 79–91.

3 For a comprehensive discussion of observability data management systems, I highly recommend Suman Karumuri, Franco Solleza, Stan Zdonik, and Nesime Tatbul's "Towards Observability Data Management at Scale", *ACM Sigmod Record* 49, no. 4 (March 2021): 18–23.